Date : 18-03-2026

Styling from elements:

Images, media and form elements:

Replaced elements:

Images and videos are described as replaced elements.

Certain replaced elements, such as images and video, are also described as having an **aspect ratio**. This means that it has a size in both the horizontal (x) and vertical (y) dimensions, and will be displayed using the intrinsic dimensions of the file by default.

Sizing Images:

If you place an image inside a box that is smaller or larger than the intrinsic dimensions of the image file in either direction, it will either appear smaller than the box, or overflow the box. You need to make a decision about what happens with the overflow.

To fit that we can use max-width : 100%;

Handling image display issue with object-fit:

As previously we used max-width: 100%; it looks that doesn't quite filled its container.

This is because giving an image a specific width causes its height to be set so that its aspect ratio is preserved.

We could set the container to have a fixed width *and* height, and then give the image a width and height of 100%.

the image is distorted as its aspect ratio has been changed — it will look *stretched.*

To fix this we can use object-fit property.

object-fit takes few different elements some are:

- cover: The image completely fills the <img> element while maintaining its aspect ratio, therefore some parts of the image are not displayed.
- contain: The image completely fits inside the <img> element while maintaining its aspect ratio, therefore some parts of the <img> element are not filled. This results in "letterboxing" or "pillarboxing".

The object-fit property resizes the image itself to fit inside the <img> element that is embedding it onto the page

The <img> needs to be resized for object-fit to have any effect.

If the <img> element is not resized, the image will be shown at its original (or intrinsic) size and aspect ratio, therefore object-fit will have no effect.

Replaced elements in layout:

When using various CSS layout techniques on replaced elements, you may well find that they behave slightly differently from other elements. For example, in a grid layout, elements are stretched by default to fill their entire grid areas. Images do not stretch; instead, they are aligned to the start of their grid areas.

Form Elements:

Many form controls are added to your page by way of the <input> element.

this defines simple form fields such as text inputs, through to more complex fields such as color and date pickers. There are some additional elements, such as <textarea> for multiline text input, and also elements used to contain and label parts of forms such as <fieldset> and <legend>.

Styling text input elements:

Elements that allow for text input such as <input type="text">, the more specific <input type="email">, and the <textarea> element, are quite easy to style and tend to behave just like other boxes on your page.

Example:

Html:

```
<form>
  <div><label for="name">Name</label> <input id="name" type="text" /></div>
  <div><label for="email">Email</label> <input id="email" type="email" /></div>

  <div class="buttons"><input type="submit" value="Submit" /></div>
</form>
```

CSS:

```
input[type="text"],
input[type="email"] {
  border: 2px solid black;
  margin-bottom: 1em;
  padding: 10px;
  width: 80%;
}
```

```
input[type="submit"] {
  border: 3px solid #333333;
  background-color: #999999;
  border-radius: 5px;
  padding: 10px 2em;
  font-weight: bold;
  color: white;
}

input[type="submit"]:hover,
input[type="submit"]:focus {
  background-color: #333333;
}
```

Normalizing form behaviour:

Form elements behave differently across different browsers and operating systems

Inheritance and form elements:

In some browsers, form elements do not inherit font styling by default. Therefore, if you want to be sure that your form fields use the font defined on the body, or on a parent element, you should add this rule to your CSS.

```
button,
input,
select,
textarea {
  font-family: inherit;
  font-size: 100%;
}
```

Form elements and box-sizing:

Across browsers, form elements use different box sizing rules for different widgets.

For consistency, it is a good idea to set margins and padding to 0 on all elements, then add these back in when styling particular controls:

```
button,
input,
select,
textarea {
  box-sizing: border-box;
  padding: 0;
  margin: 0;
}
```

In addition to the rules mentioned above, you should also set overflow: auto on <textarea> elements to stop some older browsers from showing a scrollbar when there is no need for one

```
textarea {
  overflow: auto;
}
```

Styling web forms:

Challenges in styling form widgets:

1995, the html specification introduced form controls

But CSS wasn't released until late 1996, and wasn't supported by most browsers until years afterward.

Types of widgets:

Easy-to-style:

1. <form>
2. <fieldset> and <legend>
3. Single-line text <input>s (e.g., type text, url, email), except for <input type="search">.
4. Multi-line <textarea>
5. Buttons (both <input> and <button>)
6. <label>
7. <output>

Harder-to-style:

- Checkboxes and radio buttons

- <input type="search">

Having internals can't be styled in css alone:

- <input type="color">
- Date-related controls such as <input type="datetime-local">
- <input type="range">
- <input type="file">
- Elements involved in creating dropdown widgets, including <select>, <option>, <optgroup> and <datalist>.
- <progress> and <meter>

For example, the date picker calendar, and the button on <select> that displays an options list when clicked, can't be styled using CSS alone.

<input type="color"> → Lets the user **pick a color** using a color picker.

<input type="datetime-local"> → Allows the user to **select date and time**.

<input type="range"> → Creates a **slider control** to choose a value within a range.

<input type="file"> → Lets the user **upload/select a file** from their device.

<select> → Creates a **dropdown list**.

<option> → Defines an **item inside a dropdown**.

<optgroup> → Groups related options inside a dropdown.

<datalist> → Provides **suggestions/autocomplete options** for an input field.

<progress> → Shows **progress of a task** (like loading).

<meter> → Displays a **measurement value within a range** (like battery level).

Styling simple form widgets:

Fonts and text:

some widgets do not inherit font-family and font-size from their parents

Many browsers use the system's default appearance instead. To make your forms' appearance consistent with the rest of your content, you can add the following rules to your stylesheet:

button,

input,

select,

textarea {

```
  font-family: inherit;
  font-size: 100%;
}
```

Box sizing:

All text fields have complete support for every property related to the CSS box model, such as width, height, padding, margin, and border

Box sizing rules:

```
input,
textarea,
select,
button {
  width: 150px;
  padding: 0;
  margin: 0;
  box-sizing: border-box;
}
```

Legend placement:

The <legend> element is okay to style, but it can be a bit tricky to control the placement of it. By default, it is always positioned over the top border of its <fieldset> parent, near the top left corner.

If I want place the legend somewhere else then I need to depend on the positioning.

Advanced form styling:

Appearance : controlling os level styling:

The appearance property was created as a way to control what OS- or system-level styling was applied to web form controls. By far the most helpful value, and probably the only one you'll use, is none. This stops any control you apply it to from using system-level styling, as much as possible, and lets you build up the styles yourself using CSS.

```
input {
  appearance: none;
}
```

This stops any control you apply it to from using system-level styling, as much as possible, and lets you build up the styles yourself using CSS.

Search boxes and appearances:

The appearance: none; value used to be particularly useful for consistently styling <input type="search"> elements.

Without it, Safari didn't allow height or font-size values to be set on them. However, this is no longer the case in Safari 16 and later. You may still want to target input[type="search"] explicitly with appearance: none; if your browser support matrix includes Safari versions older than 16.

In search inputs, the "x" delete button, which appears when the value is not null, disappears when the input loses focus in Edge and Chrome, but stays put in Safari. To remove via CSS, you can use this following rule:

```
input[type="search"]:not(:focus, :active)::-webkit-search-cancel-button {
  display: none;
}
```

Styling checkboxes and radio buttons using appearance:

Styling a checkbox or a radio button is tricky by default. The sizes of checkbox and radio button default styles are not meant to be changed, and browsers react very differently when you try. Some increase the size of the control, and some keep the control the same size and add extra space around it.

A much better approach is to remove the default appearance of checkboxes and radio buttons altogether with appearance: none;, and then add your own styles to their various states.

Ugly elements:

ugly" controls — the ones that are really hard to thoroughly style. In short, these are drop-down boxes, complex control types like color and datetime-local, and feedback—oriented controls like <progress> and <meter>.

The problem is that these elements have very different default looks across browsers, and while you can style them in some ways, some parts of their internals are impossible to style.

Global styles:

Selects and datalists:

Styling datalists and selects (in browsers that don't support customizable selects) allows an acceptable level of customization, provided you don't want to vary the look and feel too much from the defaults. We've managed to get the boxes looking pretty uniform and consistent. The datalist-invoking control is an <input type="text"> anyway, so we knew this wouldn't be a problem.

Two things are slightly more problematic. First of all, the select's "arrow" icon that indicates it is a dropdown differs across browsers. It also tends to change if you increase the size of the

select box or resize it in an ugly fashion. To fix this in our example, we first used our old friend appearance: none to get rid of the icon altogether:

```
Select {
        appearance: none;
}
```

We then created our own icon using generated content. We put an extra wrapper around the control, because ::before/::after don't work on <select> elements (their content is fully controlled by the browser):

Html:

```
<label for="select">Select a fruit</label>
<div class="select-wrapper">
  <select id="select" name="select">
    <option>Banana</option>
    <option>Cherry</option>
    <option>Lemon</option>
  </select>
</div>
```

CSS:

```
.select-wrapper {
  position: relative;
}

.select-wrapper::after {
  content: "▼";
  font-size: 1rem;
  top: 6px;
  right: 10px;
  position: absolute;
}
```

Date Input types:

The date/time input types (datetime-local, time, week, month) all have the same major associated issue. The actual containing box is as easy to style as any text input, and what we've got in this demo looks fine.

However, the internal parts of the control (e.g., the popup calendar that you use to pick a date, the spinner that you can use to increment/decrement values) are not stylable at all, and you can't get rid of them using appearance: none;. If you really need full control over the styling, you'll have to either use a library to generate a custom control or build your own.

Range input types:

```
<input type="range">
```

<input> elements of type **range** let the user specify a numeric value which must be no less than a given value, and no more than another given value. The precise value, however, is not considered important. This is typically represented using a slider or dial control rather than a text entry box like the number input type.

```
<div>
  <input type="range" id="volume" name="volume" min="0" max="11" />
  <label for="volume">Volume</label>
</div>
```

<input type="range"> is annoying to style. You can use something like the following to remove the default slider track completely and replace it with a custom style (a thin red track, in this case):

CSS:

```
input[type="range"] {
  appearance: none;
  background: red;
  height: 2px;
  padding: 0;
  outline: 1px solid transparent;
}
```

Color input types:

Input controls of type color are not too bad. In supporting browsers, they tend to give you a block of solid color with a small border.

```
input[type="color"] {
```

```
  border: 0;

  padding: 0;

}
```

File input type:

Inputs of type file are generally OK — as you saw in our example, it is fairly easy to create something that fits in OK with the rest of the page — the output line that is part of the control will inherit the parent font if you tell the input to do so, and you can style the custom list of file names and sizes in any way you want; we created it after all.

The only problem with file pickers is that the button you press to open the file picker is completely unstylable — it can't be sized or colored, and it won't even accept a different font.

```
input[type="file"] {

  height: 0;

  padding: 0;

  opacity: 0;

}
```

And then style the label to act like a button, which, when pressed, will open the file picker as expected:

```
label[for="file"] {

  box-shadow: 1px 1px 3px #cccccc;

  background: linear-gradient(to bottom, #eeeeee, #cccccc);

  border: 1px solid darkgrey;

  border-radius: 5px;

  text-align: center;

  line-height: 1.5;

}


label[for="file"]:hover {

  background: linear-gradient(to bottom, white, #dddddd);

}
```

```
label[for="file"]:active {
  box-shadow: inset 1px 1px 3px #cccccc;
}
```

Meters and progress bars:

we can set them to the desired width relatively accurately. But beyond that, they are really difficult to style in any way. They don't handle height settings consistently between each other and between browsers, you can color the background but not the foreground bar, and setting appearance: none on them makes things worse, not better.

Debugging CSS:

The DOM vs view source:

In the rendered DOM the browser may have normalized the HTML, for example by correcting some badly-written HTML for you. If you incorrectly closed an element, for instance by opening an <h2> but closing with an </h3>, the browser will figure out what you were meaning to do and the HTML in the DOM will correctly close the open <h2> with an </h2>. The DOM will also show any changes made by JavaScript.

View Source, in comparison, is the HTML source code as stored on the server. The HTML tree in your DevTools shows exactly what the browser is rendering at any given time, so it gives you an insight into what is really going on.

Understanding the box model:

he fact that we have an alternate box model that changes how the size of elements are calculated based on the size you give them, plus the padding and borders. DevTools can really help you to understand how the size of an element is being calculated.

Mainly here we need understand about the box-sizing.

Solving specificity issue:

Sometimes during development, but in particular when you need to edit the CSS on an existing site, you will find yourself having a hard time getting some CSS to apply. No matter what you do, the element just doesn't seem to take the CSS. What is generally happening here is that a more specific selector is overriding your changes, and here DevTools will really help you out.

Debugging problems in CSS:

Take a step back from the problem:

Think about something and get refreshed then the solution for the problem will come to mind don’t always stick to the problem do any other work for some time.

Do you have a valid html and css:

Browsers expect your CSS and HTML to be correctly written, however browsers are also very forgiving and will try their best to display your webpages even if you have errors in the markup or stylesheet. If you have mistakes in your code the browser needs to make a guess at what you meant, and it might make a different decision to what you had in mind. In addition, two different browsers might cope with the problem in two different ways. A good first step, therefore, is to run your HTML and CSS through a validator, to pick up and fix any errors.

Check are the properties and value supported by the browser you are testing in.

Browsers ignore CSS they don't understand. If the property or value you are using is not supported by the browser you are testing in then nothing will break, but that CSS won't be applied. DevTools will generally highlight unsupported properties and values in some way.

Check is something else overriding your css:

This is where the information you have learned about specificity will come into much use. If you have something more specific overriding what you are trying to do, you can enter into a very frustrating game of trying to work out what. However, as described above, DevTools will show you what CSS is applying and you can work out how to make the new selector specific enough to override it.

Make a reduced test case of the problem:

A reduced test case is a code example that demonstrates the problem in the simplest possible way, with unrelated surrounding content and styling removed. This will often mean taking the problematic code out of your layout to make a small example which only shows that code or feature.

To create a reduced test case:

1. If your markup is dynamically generated — for example via a CMS — make a static version of the output that shows the problem. A code sharing site like CodePen is useful for hosting reduced test cases, as then they are accessible online and you can easily share them with colleagues. You could start by doing View Source on the page and copying the HTML into CodePen, then grab any relevant CSS and JavaScript and include it too. After that, you can check whether the issue is still evident.

2. If removing the JavaScript does not make the issue go away, don't include the JavaScript. If removing the JavaScript *does* make the issue go away, then remove as much JavaScript as you can, leaving in whatever causes the issue.

3. Remove any HTML that does not contribute to the issue. Remove components or even main elements of the layout. Again, try to get down to the smallest amount of code that still shows the issue.

4. Remove any CSS that doesn't impact the issue.

Problems with HTML and CSS:

Bad practice:

- Using JavaScript to create entire page content and style
    - Makes site slow (poor performance)
    - Makes site inaccessible
- Using new features not supported in all browsers
    - Some users won't see styles properly
- Responsive issues
    - Website looks good on desktop but bad on mobile
    - Text too small or layout breaks
    - Heavy animations → slow loading

Fixing General problems:

Before worrying about browser issues:

- Test your site in different browsers (Chrome, Firefox, mobile, etc.)
- Check if HTML and CSS are written correctly (no syntax errors)

CSS Conflict problem:

Sometimes styles conflict with each other.

Example causes:

- Using CSS frameworks
- Using third-party code (ads, APIs)

Problem:

Same class name used in multiple places → styles clash

Solution : Namespacing (use unique class names)

.audio-player ul a {

}

This avoids conflicts.

Validation:

HTML Validation:

Check:

- Tags are properly closed
- Proper nesting
- Correct doctype
- Use correct HTML elements

For validation we can use W3C HTML Validator for validation.

CSS Validation:

Check:

- Property names are correct
- Values are valid
- Curly braces are correct

For this also we can use W3C css validator for validation.

Linters - Linters are tools that check your code for errors and bad practices before it runs.

Best to use the editor plugins

Eg: VS Code linters

Browser developer tools:

It is used to debug css and see the errors

Example:

- Invalid CSS is crossed out
- Warning icons show issues

The HTML errors are harder to see so we can use the Validator for html validation.

Common cross browser problems:

1. Browser support issues
2. HTML fallback behavior – if the browser doesn’t understands an element then it trats it like a normal inline element
    1. Fallback in media elements
       Eg: <video>
       Your browser doesn't support video
       </video> (if video not supported it shows text)
    2. Form input fallback
       Eg: <input type="date"> (if it not supported it will works like type=”text”)

3. CSS Fallback behaviour – if the browser doesn’t understands css then it simply ignores the rules.

Selector issues:

If selector is wrong then style won’t apply

If selector is invalid entire rule is ignored

Responsive design problems:

Websites should work on all devices

Common Issues:

- Layout breaks
- Text too small
- Images too large
- Slow loading

Solutions:

- Use media queries
- Use viewport
- Use responsive images (srcset, <picture>)

Use stack overflow (Stack Overflow is a popular online question-and-answer website for programmers)

CSS Text Styling:

Text and font styling:

The CSS properties used to style text generally fall into two categories, which we'll look at separately in this article:

- **Font styles**: Properties that affect a text's font, e.g., which font gets applied, its size, and whether it's bold, italic, etc.
- **Text layout styles**: Properties that affect the spacing and other layout features of the text, allowing manipulation of, for example, the space between lines and letters, and how the text is aligned within the content box.

Fonts:

These are used to assign specific font style, font-size and font related properties.

Color:

The color property sets the color of the foreground content of the selected elements, which is usually the text, but can also include a couple of other things, such as an underline or overline placed on text using the text-decoration property.

Font-families:

To set a different font for your text, you use the font-family property — this allows you to specify a font (or list of fonts) for the browser to apply to the selected elements.

The browser will only apply a font if it is available on the machine the website is being accessed on; if not, it will just use a browser default font.

Web safe fonts:

Speaking of font availability, there are only a certain number of fonts that are generally available across all systems and can therefore be used without much worry. These are the so-called **web safe fonts**.

Name	Generic type	Notes
Arial	sans-serif	It's often considered best practice to also add *Helvetica* as a preferred alternative to *Arial* as, although their font faces are almost identical, *Helvetica* is considered to have a nicer shape, even if *Arial* is more broadly available.
Courier New	monospace	Some OSes have an alternative (possibly older) version of the *Courier New* font called *Courier*. It's considered best practice to use both with *Courier New* as the preferred alternative.
Georgia	serif	
Times New Roman	serif	Some OSes have an alternative (possibly older) version of the *Times New Roman* font called *Times*. It's considered best practice to use both with *Times New Roman* as the preferred alternative.
Trebuchet MS	sans-serif	You should be careful with using this font — it isn't widely available on mobile OSes.
Verdana	sans-serif	

Default fonts:

CSS defines five generic names for fonts: serif, sans-serif, monospace, cursive, and fantasy.

These are very generic and the exact font face used from these generic names can vary between each browser and each operating system that they are displayed on.

serif, sans-serif, and monospace are quite predictable and should provide something reasonable. On the other hand, cursive and fantasy are less predictable and we recommend using them very carefully, testing as you go.